Power Spectral Density of Line Code

TEAM 01

Youssef Mohammed Ibrahim Sayed

Youssef Hisham Ahmed Hikel

Mina Wael Tanagho Fahmy

Nader Hany Maurice Wahib

Abdelrahaman Ahmed Kamal Atta

Faculty of Engineering
Department of Electronics and Communications
Course code : EECG232-17
Course name : Introduction to Communication Systems

Power Spectral Density of Line Code

TEAM 01

يوسف محمد ابراهيم سيد

Student ID : 91240892
Section : 4
BN : 34

يوسف هشام أحمد هيكل

Student ID : 91240907
Section : 4
BN : 36

مينا وائل تناغو فهمي

Student ID : 91240804
Section : 4
BN : 14

نادر هاني موريس وهيب

Student ID : 91240805
Section : 4
BN : 15

عبدالرحمن أحمد كمال عطا

Student ID : 91240410
Section : 2
BN : 24

Supervisor: Dr. Mohammed Nafie
Professor, Cairo University

Co-supervisor: ENG. Mohammed Khaled
Teacher Assistant, Cairo University

Faculty of Engineering

Department of Electronics and Communications

Course code : EECG232-17

Course name : Introduction to Communication Systems

Abstract

This project explores the implementation of digital communication systems using the software radio technique. By shifting traditional hardware-based radio problems into the software domain, we simulate a transmitter and receiver environment that closely mirrors real-world antenna behavior. The primary focus is the generation and analysis of three specific line codes: Polar RZ, Polar NRZ, and Unipolar.

Since the transmitted bit-streams and initial timing offsets are inherently random, the system is modeled as a random process. Using MATLAB, we generate an ensemble of realizations for each signaling technique to evaluate their statistical and time-domain characteristics. Specifically, we analyze the statistical mean, time mean, autocorrelation functions, and power spectral density. These results are then used to verify the stationarity and ergodicity of the processes, providing a comprehensive understanding of how software-defined waveforms behave in a practical communications environment.

Contents

1 Problem Description **2**

2 Introduction **3**

3 Control Flags **4**

4 Generation of Data **5**

5 Creating the Unipolar Ensemble **6**

6 Creating The Polar NRZ Ensemble **8**

7 Creating The Polar RZ Ensemble **9**

8 Q1 Statistical Mean Calculation **10**

9 Q2 Is the Process Stationary **13**

10 Q3 Statistical Autocorrelation **14**

11 Q4 Time Mean & Autocorrelation of One Waveform **15**

12 Q5 Is the Random Process Ergodic **16**

13 Plotting the PSD of the Ensemble **17**

14 Q6 Signal Bandwidth **18**

Appendices

A Appendix **20**

References 21

1

Problem Description

In this project, we are simulating a software radio. Software radio is a technique that uses software to perform radio signal processing tasks that were traditionally done by hardware. The software radio allows for greater flexibility and adaptability in radio communication systems, as it can be easily reconfigured and updated through software changes. In this project, we will be simulating a software radio using MATLAB. We will be implementing various components of the software radio, such as modulation, demodulation, filtering, and channel coding, to create a complete simulation of a software radio system and see the problems ourselves and try design a proper solution.

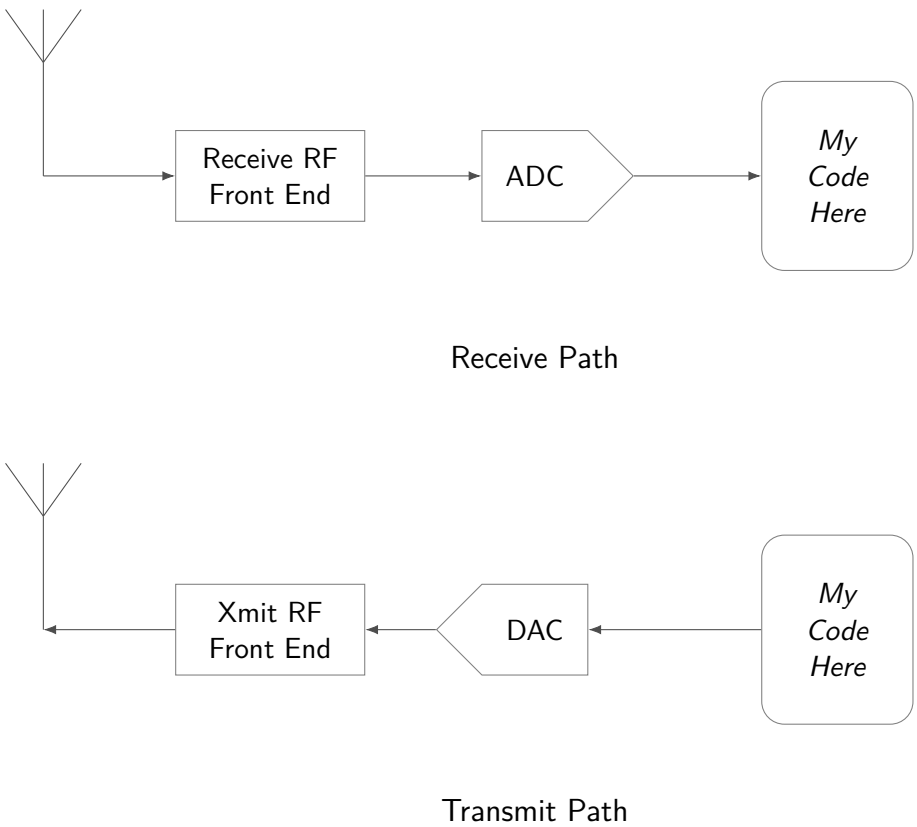

Figure 1.1: *Software Radio Transmit and Receive Paths.*

2

Introduction

In this project, we are going to use the software radio technique to simulate the hardware problems and try to make it as close as it is with the antenna, while also using the techniques we learned through our communications course. We are going to design the transmitter first. Our code at the transmitter will define the modulation technique and the coding used. Once the digital data is generated, it goes through a Digital-to-Analog Converter (DAC) so it can actually take the form of three different transmitted signals: Polar RZ, Polar NRZ, and Unipolar. However, the bit-stream generated at the transmitter is actually random, and there can also be asynchronization between the transmitter and the receiver, which is also random. Therefore, in our simulation, we will generate a random bit-stream, and the output will be treated as a random process. Then, for each generated ensemble, we will calculate its statistical mean, the time mean, the ensemble and time autocorrelation functions, and the bandwidth. We will also determine whether the random process is stationary and ergodic.

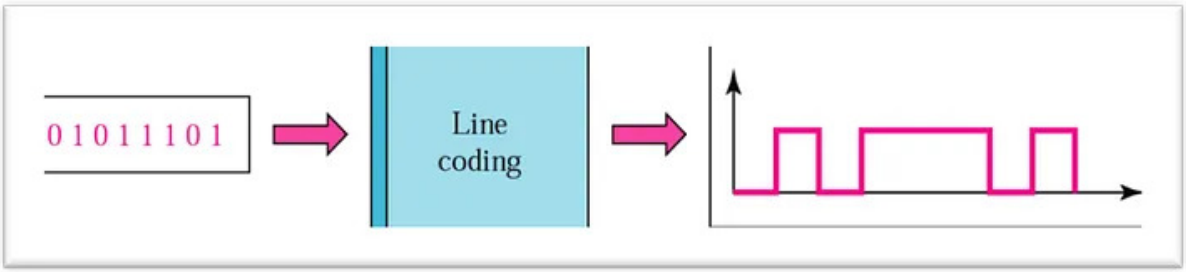

Figure 2.1: *A simple illustration of the line coding process.*

3

Control Flags

Listing 1: *Defining the control flags.*

```
%% Control Flags
N_bits         = 100;           % Number of bits per realization.
N_realizations = 500;           % Number of realizations.
Pw             = 0.07;          % Pulse width of the bit.
Ts             = 0.01;          % Time sample.
L              = round(Pw/Ts);  % Samples per symbol period (L=7).
A              = 4;             % Amplitude.
N_fft          = 1024;          % FFT size.
Fs             = 1/Ts;          % Sampling frequency.
freq           = (-N_fft/2:N_fft/2-1) * (Fs/N_fft);  % Frequency axis.
center         = floor(N_fft/2) + 1;                  % Center bin.
half_len       = 100;           % Max tau used in correlation_manual.
```

- `N_bits`: Defines the number of bits generated in each realization of the random process. It controls the length of the bit stream.
- `N_realizations`: Specifies the number of independent realizations of the random process.
- `Pw`: Represents the pulse width (bit duration) in seconds. It determines the time-domain duration of each bit and directly affects the signal's bandwidth.
- `Ts`: The time sampling interval in seconds.
- `L`: It determines how many discrete samples represent a single bit.
- `A`: Sets the peak amplitude of the transmitted signal.
- `N_fft`: Specifies the size of the Fast Fourier Transform (FFT).
- `Fs`: The sampling frequency $(1/Ts)$. It defines the frequency range of the simulation and must be high enough to satisfy the Nyquist criterion.
- `freq`: The frequency axis vector used for spectral analysis, mapped from $-Fs/2$ to $Fs/2$ to allow for centered Power Spectral Density (PSD) plots.
- `center`: The index of the zero-frequency (DC) component in the FFT output, used to correctly align and shift the spectrum for plotting.
- `half_len`: Defines the maximum lag (τ) for the manual calculation of the autocorrelation function.

4

Generation of Data

Now that the control flags are defined, we can proceed with the first step of the simulation: generating the raw binary data. To accurately simulate a real-world communication source, we use the MATLAB function `randi` to produce a random bit-stream.

Listing 2: *Generation of Data.*

```
%% Data Generation
data = randi([0 1], N_realizations, N_bits);
```

The function `randi([0 1],` $N_{\texttt{realizations}}$`,` $N_{\texttt{bits}}$`)` generates a matrix of independent random integers. In this setup, each row represents a unique *realization* of the random process, while each column represents a *bit* in the sequence.

This matrix structure is fundamental to our analysis; it allows us to treat the entire project as a discrete ensemble. By performing operations across the rows, we can calculate ensemble statistics (like the statistical mean), and by operating across the columns, we can determine time-domain characteristics (like the time average). Furthermore, using a range of $[0, 1]$ ensures an equiprobable source where the probability of a "0" or a "1" is exactly 0.5, providing a neutral baseline for comparing the efficiency of different line codes.

Realizations

Realization 1	1	0	1	0	1	1	0	1	0	1	0	0	0	0	1	0	1	0	0	1	0	1	1	0
Realization 2	1	0	0	1	0	1	0	1	0	1	0	1	1	1	1	1	1	0	0	0	0	0	0	0
Realization 3	0	1	1	0	1	0	1	0	1	0	0	0	0	0	0	1	1	1	1	0	1	0	1	0
Realization 5	1	0	0	1	0	1	1	1	1	1	1	1	0	0	0	1	1	1	0	0	1	1	1	1
Realization 1000	0	1	0	1	0	0	1	1	1	1	0	0	0	0	0	0	1	1	1	0	1	1	0	0
Realization 9999	1	0	1	0	0	0	0	1	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0
Realization 10^9	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Realization ∞	1	1	0	1	0	0	0	1	0	1	0	0	1	1	1	1	1	1	1	1	1	0	0	0

t

Figure 4.1: *A visual example of the generated data matrix.*

5

Creating the Unipolar Ensemble

Now that we have defined the control flags and generated the data, we can proceed to create our signals, we will start with the unipolar line code, which might be the simplest among the three line codes we are going to analyze. But first, lets create the ensemble. As we are going to analyze multiple codes we are going to create a function that will create the ensemble then use it to create the ensemble for each code.

Listing 3: *Creating Ensemble function.*

```
function ensample = build_ensemble(data, A, pulse, N_realizations, L, type)
if strcmp(type, 'unipolar'), Tx = data * A;          % Map bits to {0, A}.
else,                        Tx = (2*data - 1) * A;  % Map bits to {-A, +A}.
end
ensample = kron(Tx, pulse);                          % Upsample with pulse shape.
Td = randi([0, L-1], N_realizations, 1);             % Random delay per realization.
for i = 1:N_realizations
    ensample(i,:) = circshift(ensample(i,:), [0, Td(i)]);
end
end
```

The function `build_ensemble` takes the raw binary data and transforms it into a matrix of time-domain signals, where each row corresponds to a different realization of the random process. Also a random time delay is added using `randi` function, then `circshift` is used to apply the random delay to each realization, ensuring that each signal starts at a different time instant.

- `data` parameter is the generated random bits from Listing 3. These data will be mapped to 0 and A for unipolar, and to $-A$ and $+A$ for polar.
- `pulse` parameter defines the shape of the pulse used for upsampling, which in our case is a rectangular pulse of width L.
- `N_realizations` specifies how many independent signal realizations we want to generate, which corresponds to the number of rows in the output matrix.
- `L` is the number of samples per symbol period, which determines how the bits are upsampled in time.
- `type` is a string parameter that specifies the line code type, allowing the function to handle both unipolar and polar mappings and distinguish between them when generating the ensemble.

Now that we created the function to create the ensemble, we can use it to create the unipolar ensemble that we aimed to create initially in this part of the project.

Listing 4: *Creating the Unipolar Ensemble.*

```
%% Unipolar NRZ
fprintf('\nUnipolar NRZ\n');
pulse_unipolar  = ones(1, L); % Pulse shape for unipolar NRZ (rectangular pulse).
ylim_unipolar   = [-1, A+1]; % Set y-axis limits for the plot.
color_unipolar  = [0.8500 0.3250 0.0980]; % Orange color for the plot.

ensample_unipolar = build_ensemble(data, A, pulse_unipolar, N_realizations, L, 'unipolar');

```

All we did in Listing 4 is setting the pulse parameter and passing all the existing parameters to the `build_ensemble` function, for the data we used the generated data from Listing 3, for `A`, $N_{realizations}$, `L` we used the defined amplitude, number of realizations, and Samples per symbol period from the control flags in Listing 1.

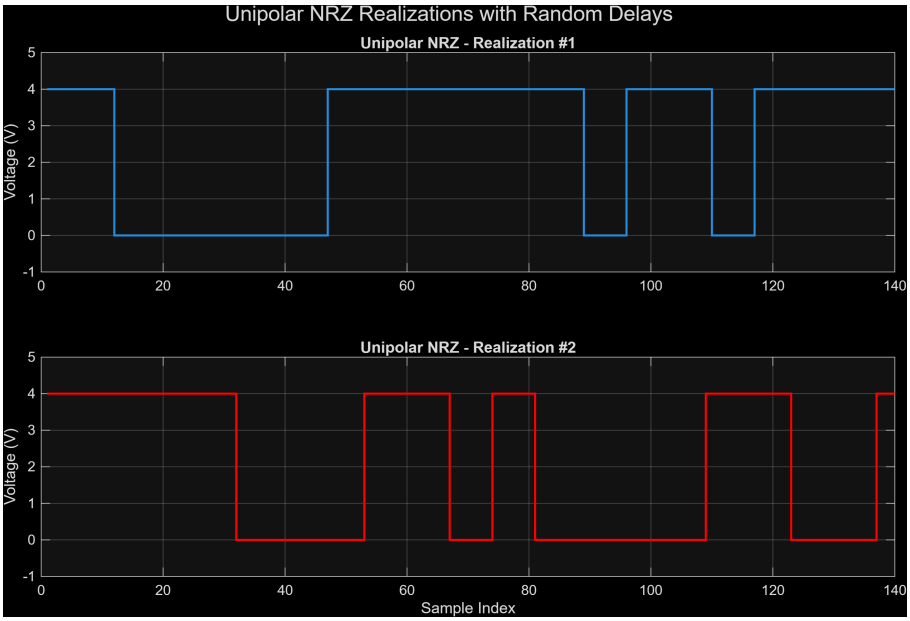

Figure 5.1: *Realization 1 and Realzation 2 from the Unipolar Ensemble.*

Now we can see in Figure 5.1 the first two realizations of the unipolar ensemble, the signal is a rectangular pulse as we expected from the input pulse, and the amplitude maps to 0 and 4 as how we assigned A earlier in the control flags. This output confirms that we succefully created the unipolar ensemble as we met the required specifications for the unipolar line code.

- The amplitude of the signal is either 0 (for bit "0") or A (for bit "1").
- The pulse shape is rectangular, with a duration of L samples per bit.

6

Creating The Polar NRZ Ensemble

Similar to what we did in the previous chapter when we were creating the unipolar ensemble, we will apply the same steps to create the polar NRZ ensemble.

Listing 5: *Creating the Polar NRZ Ensemble.*

```
%% Polar NRZ
fprintf('\nPolar NRZ\n');
pulse_polar_NRZ = ones(1, L);
ylim_polar_NRZ  = [-A-1, A+1];
color_polar_NRZ = [0 0.4470 0.7410];

ensample_polar_NRZ = build_ensemble(data, A, pulse_polar_NRZ, N_realizations, L, 'polar');
```

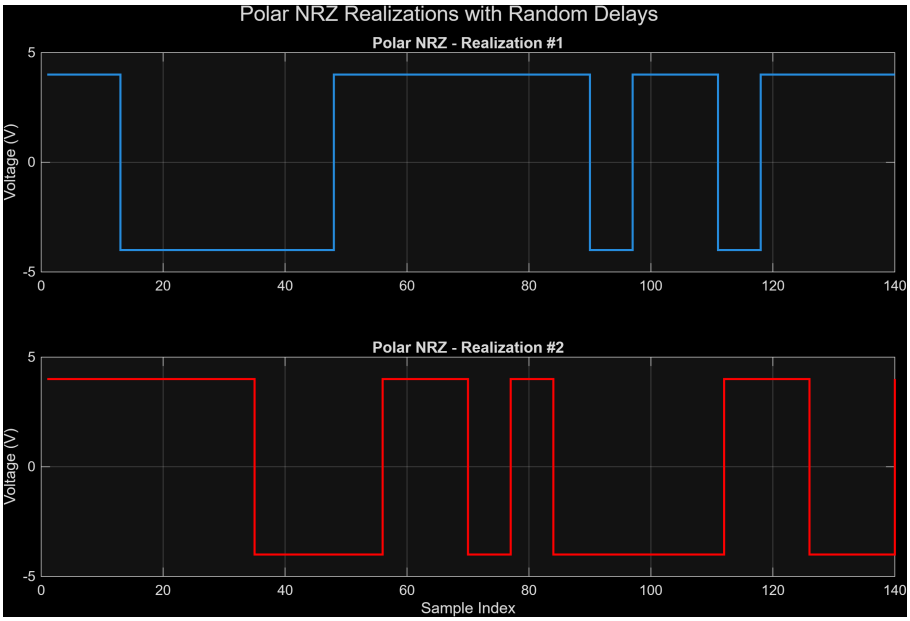

Figure 6.1: *Realization 1 and Realzation 2 from the Polar NRZ Ensemble.*

As we can see in Figure 6.1

- The signal is either at $-A$ or at $+A$, which is the expected behavior for the polar NRZ line code.
- The pulse shape is rectangular, with a duration of L samples per bit.

7

Creating The Polar RZ Ensemble

Nothing changed or to say here, we will just apply the same steps again.

Listing 6: *Creating the Polar RZ Ensemble.*

```
%% Polar RZ
fprintf('\nPolar RZ\n');
pulse_RZ = [ones(1, floor(L/2)), zeros(1, ceil(L/2))];
ylim_RZ  = [-A-1, A+1];
color_RZ = [0.4940 0.1840 0.5560];

ensample_RZ = build_ensemble(data, A, pulse_RZ, N_realizations, L, 'polar');
```

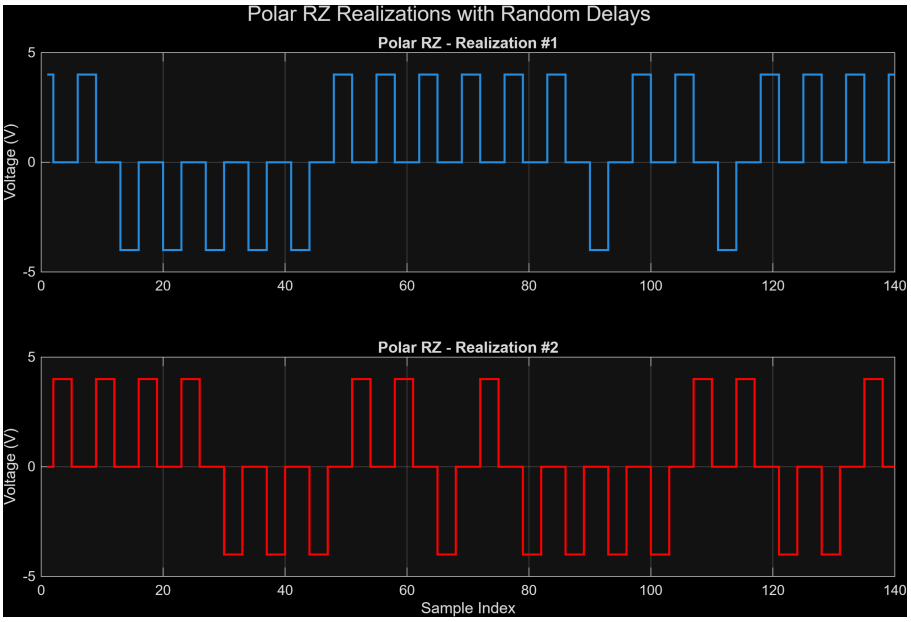

Figure 7.1: *Realization 1 and Realzation 2 from the Polar RZ Ensemble.*

As we can see in Figure 7.1

- The signal is now a pulse that lasts for half of the bit duration, and then goes to zero for the other half. This is the main characteristic of RZ signals.
- The pulse shape is rectangular, with a duration of L samples per bit.

8

Q1 Statistical Mean Calculation

First, we will create a function that can caluclate the mean of the ensemble of any signal we generated.

Listing 7: *Manual Mean Calculation Function.*

```
function mean_val = manual_mean(data, dim)
if nargin == 1,    mean_val = sum(data, 'all') / numel(data);  % All elements.
elseif dim == 1,   mean_val = sum(data, 1) / size(data, 1);    % Column-wise (ensemble).
elseif dim == 2,   mean_val = sum(data, 2) / size(data, 2);    % Row-wise (time).
end
end
```

The function `manual_mean` is designed to compute the mean of any signal passed to it,

The logic of the function is designed as how we were taught in the course on how to calculate the mean specfically in [1]

$$\mu = \sum_{i=1}^{N} p(x_i)x_i \tag{8.1}$$

Equation 8.1 shows the general expression for calculating the mean of a random variable, now since the $p(x_i)$ is always the same for all bits, so we can simplify the equation to be:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i \tag{8.2}$$

In Listing 7 we have implemented Equation 8.2 to calculate the mean of any input to this function, then we adjusted the input parameters to be able to calculate the mean either for the ensemble or for the time samples.

- If the user does not specify the dimenstion, the function will calculate the mean of all the elements in the input data.
- If the user specifies the dimension as 1, the function will calculate the mean column-wise, which is the mean of the ensemble for each time sample.
- If the user specifies the dimension as 2, the function will calculate the mean row-wise, which is the mean of the time samples for each ensemble member.

Now after creating the functio to calculate the mean manually, we will create another function to make the code easier and more readable, its only purpose is to use the `manual_mean` and make the code less messy.

Listing 8: *Mean Calculation Wrapper Function.*

```
        function [Ex, Ux] = compute_means(ensample, theoretical_mean)
Ex = manual_mean(ensample, 1);   % Ensemble mean (across realizations).
Ux = manual_mean(ensample, 2);   % Time mean (across samples).
fprintf('  Sample 10 ensemble mean  = %.4f V (Theoretical: %.2f V)\n',
↪  manual_mean(ensample(:,10)),  theoretical_mean);
fprintf('  Realization 15 time mean = %.4f V (Theoretical: %.2f V)\n',
↪  manual_mean(ensample(15,:)), theoretical_mean);
end
```

The `compute_means` function takes the ensemble as an argument, also we pass the theoretical mean as an argument to this function to compare the output of the ensemble to the theortical mean calculated by the techniques learned in the course. Now after creating the two functions needed to claculate the mean easily and in an organized order, we can use these functions to calculate the mean of each ensemble we created in one line of code.

Listing 9: *Mean Calculation for Unipolar*

```
    [Ex_unipolar, Ux_unipolar] = compute_means(ensample_unipolar, A/2);
```

Listing 10: *Mean Calculation for Polar NRZ*

```
    [Ex_polar_NRZ, Ux_polar_NRZ] = compute_means(ensample_polar_NRZ, 0);
```

Listing 11: *Mean Calculation for Polar RZ*

```
    [Ex_RZ, Ux_RZ] = compute_means(ensample_RZ, 0);
```

Now to calculate the theortical mean for each ensemble across realizations we can use Equation 8.2.

- For the unipolar ensemble, since the bits are represented by A and 0, the mean can be calculated as:
$$\mu_{unipolar} = \frac{1}{2}(A + 0) = \frac{A}{2} \tag{8.3}$$
- For the Polar NRZ ensemble, since the bits are represented by A and $-A$, the mean can be calculated as:
$$\mu_{polar_NRZ} = \frac{1}{2}(A + (-A)) = 0 \tag{8.4}$$

- For the Polar RZ ensemble, since the bits are represented by A, $-A$ and 0, the mean can be calculated as:

$$\mu_{polar_RZ} = \frac{1}{3}(A + (-A) + 0) = 0 \tag{8.5}$$

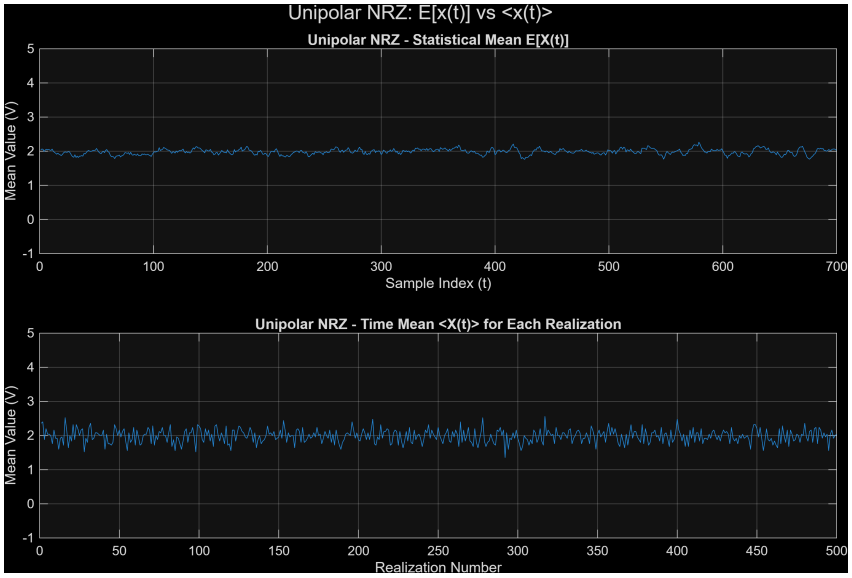

Figure 8.1: *UNIPOLAR NRZ mean across time and across realizations.*

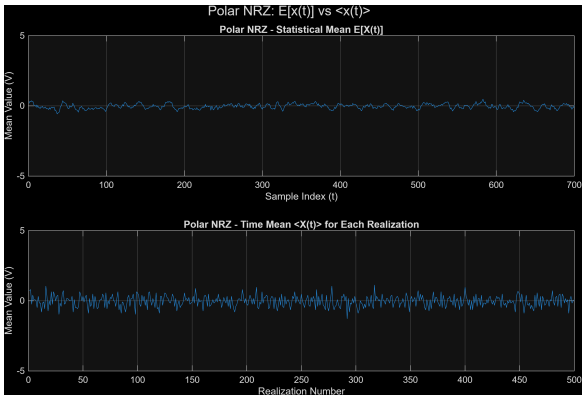

Figure 8.2: *POLAR NRZ mean across time and across realizations.*

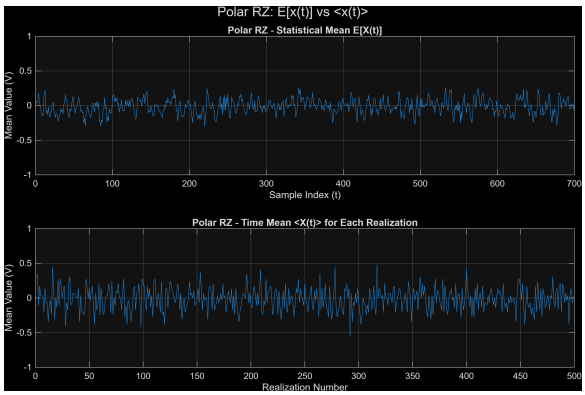

Figure 8.3: *POLAR RZ mean across time and across realizations.*

- In Figure 8.1 we can see that the mean across realizations is around $A/2$ which is nearly the theoretical mean calculated in Equation 8.3
- In Figure 8.2 we can see that the mean across realizations is around 0 which is nearly the theoretical mean calculated in Equation 8.4
- In Figure 8.3 we can see that the mean across realizations is around 0 which is nearly the theoretical mean calculated in Equation 8.5

9

Q2 Is the Process Stationary

10

Q3 Statistical Autocorrelation

11

Q4 Time Mean & Autocorrelation of One Waveform

12

Q5 Is the Random Process Ergodic

13

Plotting the PSD of the Ensemble

14

Q6 Signal Bandwidth

Appendices

A

Appendix

References

[1] D. M. Nafie, *Digital communications lecture 2*, Mar. 2026.

[2] D. M. Nafie, *Digital communications lecture 3*, Mar. 2026.